Graph and Modern Database

Implementation and Analysis of NoSQL Databases: Redis, Cassandra, JSON Schema, and MongoDB

Student Name: Shekhar Yadav

Student ID:001496957

VM Number: COMP1835-NOSQL-090

Module Leader: Dr. Margarita Razgon

Tutor: Dr. Hai Huang

Assignment 1-Redis Database

Implementation

1.Case Study

For this assignment, a Redis database is created to represent an Online Bookstore System. This system will hold data about the bookstore, books, orders, categories, ratings, and sessions.

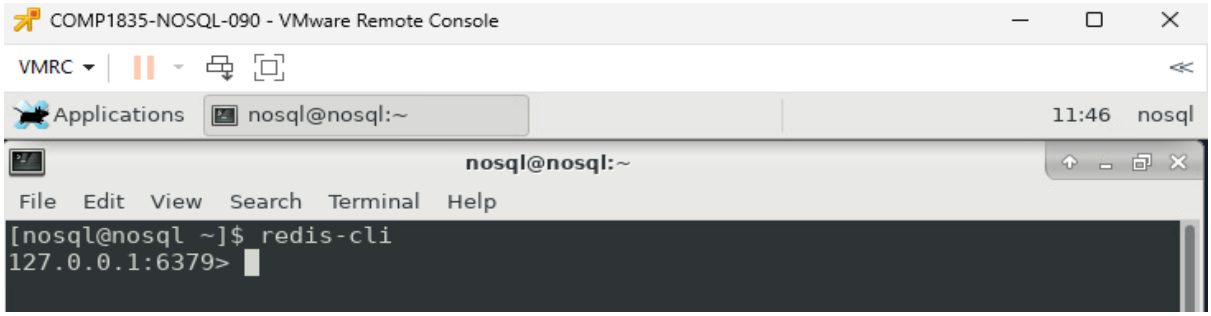
Redis is chosen for this database due to its support for various data structures and high-performance in-memory storage. This is important for fast data retrieval.

This database will show various data types supported by Redis, including:

- Strings
- Hashes
- Lists
- Sets
- Sorted Sets
- Expiring Keys

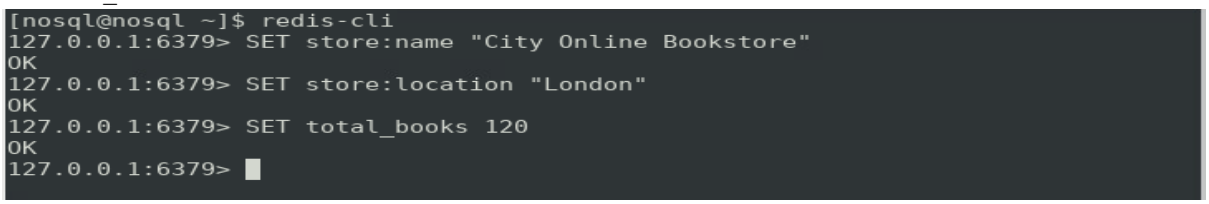
2.Database Creation

The database was created using Redis CLI by inserting key-value pairs representing different entities of the bookstore system.



Store information (String)

SET store:name "City Online Bookstore"
SET store:location "London"
SET total_books 120



Book Information (Hashes)

HSET book:101 title "Redis Essentials" author "Maxwell Dayvson" price 29.99 stock 15
HSET book:102 title "Learning NoSQL" author "John Smith" price 35.50 stock 10
HSET book:103 title "MongoDB Basics" author "Sarah Connor" price 27.75 stock 20

```
127.0.0.1:6379> HSET book:101 title "Redis Essentials" author "Maxwell Dayvson"
price 29.99 stock 15
(integer) 4
127.0.0.1:6379> HSET book:102 title "Learning NoSQL" author "John Smith" price 3
5.50 stock 10
(integer) 4
127.0.0.1:6379> HSET book:103 title "MongoDB Basics" author "Sarah Connor" price
 27.75 stock 20
(integer) 4
127.0.0.1:6379>
```

Order (List)

Lists were used to store customer orders.

LPUSH orders "order1001"
LPUSH orders "order1002"
LPUSH orders "order1003"

```
127.0.0.1:6379> LPUSH orders "order1001"
(integer) 1
127.0.0.1:6379> LPUSH orders "order1002"
(integer) 2
127.0.0.1:6379> LPUSH orders "order1003"
(integer) 3
```

Book Categories (Set)

Sets were used to store unique categories.

SADD categories "Database"
SADD categories "Programming"
SADD categories "Technology"

```
127.0.0.1:6379> SADD categories "Database"
(integer) 1
127.0.0.1:6379> SADD categories "Programming"
(integer) 1
127.0.0.1:6379> SADD categories "Technology"
(integer) 1
```

Student Marks Ranking (Sorted Sets)

Another sorted set was created to demonstrate ranking functionality.

ZADD student:marks 85 "Alice"
ZADD student:marks 92 "Bob"
ZADD student:marks 78 "Charlie"
ZADD student:marks 88 "David"
ZADD student:marks 95 "Emma"

```
127.0.0.1:6379> ZADD student:marks 85 "Alice"
(integer) 1
127.0.0.1:6379> ZADD student:marks 92 "Bob"
(integer) 1
127.0.0.1:6379> ZADD student:marks 78 "Charlie"
(integer) 1
127.0.0.1:6379> ZADD student:marks 88 "David"
(integer) 1
127.0.0.1:6379> ZADD student:marks 95 "Emma"
(integer) 1
```

User Session with Expiration

Redis support automatic expiration of keys.

SET user:session:1001 "active"
EXPIRE user:session:1001 3600

```
127.0.0.1:6379> SET user:session:1001 "active"
OK
127.0.0.1:6379> EXPIRE user:session:1001 3600
(integer) 1
```

3.Executing Queries

QUERY 1: Retrieve data:

```
127.0.0.1:6379> MGET store:name store:location
1) "City Online Bookstore"
2) "London"
127.0.0.1:6379> get total_books
"120"
127.0.0.1:6379>
```

QUERY 2 & 3: Retrieve full book information and Retrieve selected fields

```
127.0.0.1:6379> HGETALL book:101
1) "title"
2) "Redis Essentials"
3) "author"
4) "Maxwell Dayvson"
5) "price"
6) "29.99"
7) "stock"
8) "15"
127.0.0.1:6379> HGET book:102 title price
(error) ERR wrong number of arguments for 'hget' command
127.0.0.1:6379> hmget book:102 title price
1) "Learning NoSQL"
2) "35.50"
```

QUERY 4: Update book stock

```
127.0.0.1:6379> HINCRBY book:101 stock -1
(integer) 14
```

QUERY 5: View order history

```
127.0.0.1:6379> lrange orders 0 -1
1) "order1003"
2) "order1002"
3) "order1001"
```

QUERY 6: View all categories

```
127.0.0.1:6379> smembers categories
1) "Technology"
2) "Programming"
3) "Database"
127.0.0.1:6379>
```

QUERY 7 & 8: Retrieve leaderboard and Top performers

```
127.0.0.1:6379> zrange student:marks 0 -1 withscores
 1) "Charlie"
 2) "78"
 3) "Alice"
 4) "85"
 5) "David"
 6) "88"
 7) "Bob"
 8) "92"
 9) "Emma"
10) "95"
127.0.0.1:6379> ZREVRANGE student:marks 0 2 WITHSCORES
1) "Emma"
2) "95"
3) "Bob"
4) "92"
5) "David"
6) "88"
127.0.0.1:6379>
```

QUERY 9: Count students with marks above 80

```
127.0.0.1:6379> ZCOUNT student:marks 80 100
(integer) 4
```

QUERY 10: Remove students below pass mark

```
127.0.0.1:6379> ZREMRANGEBYSCORE student:marks 0 79
(integer) 1
```

QUERY 11: Check session

```
127.0.0.1:6379> GET user:session:1001
"active"
127.0.0.1:6379> TTL user:session:1001
(integer) 3399
```

These queries demonstrate filtering, ranking, counting, and dynamic updates within the Redis database.

4.Design Rational

The database design highlights Redis data structures' advantages:

Data Structure	Purpose
String	Store simple store information
Hashes	Store structured book data
Lists	Maintain order history
Sets	Store unique categories
Sorted Sets	Manage ratings and rankings
Expiring Keys	Handle user sessions

5. Benefits of Using Redis

- Because of the in-memory storage, read and write operations are incredibly quick.
- Adaptable data structures for various data kinds.
- basic key-value system.
- built-in scoring and ranking using sorted sets.
- Automatic key expiration is supported.

6. Drawbacks

- Data is stored in memory.
- It requires more RAM to store large amounts of data.
- It is more difficult to run complex relational queries than in relational databases.
- Persistence configuration is necessary to avoid data loss.

7. Implementation Evidence

Screenshots were taken to demonstrate successful execution of the Redis commands including:

- Database creation commands
- Data insertion
- Query execution
- Retrieval of stored data

- Sorted set rankings
- Session expiration checks

These screenshots confirm that the Redis database was successfully implemented.

Assignment 2 – Cassandra Database

Introduction to Cassandra

Apache Cassandra is a highly scalable distributed NoSQL database used to store huge amounts of structured data across many machines. It was created by Facebook and is now managed by the Apache Software Foundation.

Apache Cassandra is based on a peer-to-peer model, meaning all the nodes in the database are treated equally and no single node can cause a failure. This makes Cassandra a highly reliable database as all the data will be replicated across all the nodes.

Apache Cassandra does not follow the traditional database model as it stores data in a column family model. In Cassandra, data is stored in tables called column **families.** This model makes Cassandra different from traditional database systems as Cassandra can store huge amounts of data and process queries at high speeds.

One of the key features of Cassandra is that it supports horizontal scalability, meaning more nodes can be added to the database system without shutting down the database. This feature makes Cassandra suitable for applications that need to store huge amounts of data.

In this assignment, Cassandra will be used to store data in an online bookstore database. This will demonstrate the ability of Cassandra to store huge amounts of structured data.

Start Cassandra

```
[nosql@nosql ~]$ cqlsh
Connected to Test Cluster at 127.0.0.1:9042.
[cqlsh 5.0.1 | Cassandra 3.11.13 | CQL spec 3.4.4 | Native protocol v4]
Use HELP for help.
cqlsh>
```

Create Keyspace (Database)

```
cqlsh> CREATE KEYSPACE bookstore
   ... WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};
```

Create Table

```
cqlsh> USE bookstore;
cqlsh:bookstore> CREATE TABLE orders (
             ...     order_id INT PRIMARY KEY,
             ...     customer_name TEXT,
             ...     book_title TEXT,
             ...     category TEXT,
             ...     price FLOAT,
             ...     quantity INT,
             ...     order_date DATE,
             ...     status TEXT
             ... );
cqlsh:bookstore>
```

Insert Data

```
cqlsh:bookstore> INSERT INTO orders (order_id, customer_name, book_title, catego
ry, price, quantity, order_date, status) VALUES (1, 'Alice', 'Redis Essentials',
 'Database', 29.99, 1, '2026-03-01', 'Delivered');
s
cqlsh:bookstore> INSERT INTO orders (order_id, customer_name, book_title, catego
ry, price, quantity, order_date, status) VALUES (2, 'Bob', 'Learning NoSQL', 'Da
tabase', 35.50, 2, '2026-03-02', 'Shipped');
r
cqlsh:bookstore> INSERT INTO orders (order_id, customer_name, book_title, catego
ry, price, quantity, order_date, status) VALUES (3, 'Charlie', 'MongoDB Basics',
 'Database', 27.75, 1, '2026-03-03', 'Pending');
i
cqlsh:bookstore> INSERT INTO orders (order_id, customer_name, book_title, catego
ry, price, quantity, order_date, status) VALUES (4, 'David', 'Python Programming
', 'Programming', 40.00, 1, '2026-03-04', 'Delivered');
T
cqlsh:bookstore> INSERT INTO orders (order_id, customer_name, book_title, catego
ry, price, quantity, order_date, status) VALUES (5, 'Emma', 'Java Fundamentals',
 'Programming', 45.00, 3, '2026-03-05', 'Cancelled');
R
cqlsh:bookstore>
cqlsh:bookstore> INSERT INTO orders (order_id, customer_name, book_title, catego
ry, price, quantity, order_date, status) VALUES (6, 'Frank', 'Data Science Intro
', 'Technology', 50.00, 1, '2026-03-06', 'Delivered');
```

```
cqlsh:bookstore> INSERT INTO orders (order_id, customer_name, book_title, catego
ry, price, quantity, order_date, status) VALUES (7, 'Grace', 'AI Basics', 'Techn
ology', 60.00, 2, '2026-03-07', 'Shipped');
cqlsh:bookstore> INSERT INTO orders (order_id, customer_name, book_title, catego
ry, price, quantity, order_date, status) VALUES (8, 'Henry', 'Cloud Computing',
'Technology', 55.00, 1, '2026-03-08', 'Pending');
cqlsh:bookstore> INSERT INTO orders (order_id, customer_name, book_title, catego
ry, price, quantity, order_date, status) VALUES (9, 'Ivy', 'SQL Mastery', 'Datab
ase', 38.00, 2, '2026-03-09', 'Delivered');
cqlsh:bookstore> INSERT INTO orders (order_id, customer_name, book_title, catego
ry, price, quantity, order_date, status) VALUES (10, 'Jack', 'Big Data Analytics
', 'Technology', 70.00, 1, '2026-03-10', 'Shipped');
cqlsh:bookstore>
cqlsh:bookstore> INSERT INTO orders (order_id, customer_name, book_title, catego
ry, price, quantity, order_date, status) VALUES (11, 'Alice', 'Machine Learning'
, 'Technology', 65.00, 1, '2026-03-11', 'Delivered');
cqlsh:bookstore> INSERT INTO orders (order_id, customer_name, book_title, catego
ry, price, quantity, order_date, status) VALUES (12, 'Bob', 'Deep Learning', 'Te
chnology', 75.00, 1, '2026-03-12', 'Pending');
cqlsh:bookstore> INSERT INTO orders (order_id, customer_name, book_title, catego
ry, price, quantity, order_date, status) VALUES (13, 'Charlie', 'C++ Basics', 'P
rogramming', 30.00, 2, '2026-03-13', 'Delivered');
```

```
cqlsh:bookstore> INSERT INTO orders (order_id, customer_name, book_title, catego
ry, price, quantity, order_date, status) VALUES (14, 'David', 'JavaScript Guide'
, 'Programming', 35.00, 1, '2026-03-14', 'Shipped');
cqlsh:bookstore> INSERT INTO orders (order_id, customer_name, book_title, catego
ry, price, quantity, order_date, status) VALUES (15, 'Emma', 'Web Development',
'Programming', 45.00, 2, '2026-03-15', 'Delivered');
cqlsh:bookstore>
cqlsh:bookstore> INSERT INTO orders (order_id, customer_name, book_title, catego
ry, price, quantity, order_date, status) VALUES (16, 'Frank', 'Cyber Security',
'Technology', 80.00, 1, '2026-03-16', 'Pending');
cqlsh:bookstore> INSERT INTO orders (order_id, customer_name, book_title, catego
ry, price, quantity, order_date, status) VALUES (17, 'Grace', 'Blockchain Basics
', 'Technology', 90.00, 1, '2026-03-17', 'Delivered');
cqlsh:bookstore> INSERT INTO orders (order_id, customer_name, book_title, catego
ry, price, quantity, order_date, status) VALUES (18, 'Henry', 'Networking Essent
ials', 'Technology', 50.00, 2, '2026-03-18', 'Shipped');
cqlsh:bookstore> INSERT INTO orders (order_id, customer_name, book_title, catego
ry, price, quantity, order_date, status) VALUES (19, 'Ivy', 'Data Mining', 'Data
base', 60.00, 1, '2026-03-19', 'Delivered');
cqlsh:bookstore> INSERT INTO orders (order_id, customer_name, book_title, catego
ry, price, quantity, order_date, status) VALUES (20, 'Jack', 'Operating Systems'
, 'Technology', 55.00, 2, '2026-03-20', 'Cancelled');
```

Queries

Query 1 – View all data

```
cqlsh:bookstore>
cqlsh:bookstore> SELECT * FROM orders;

 order_id | book_title            | category    | customer_name | order_date | price | quantity | status
----------+-----------------------+-------------+---------------+------------+-------+----------+-----------
        5 |     Java Fundamentals | Programming |          Emma | 2026-03-05 |    45 |        3 | Cancelled
       10 |    Big Data Analytics |  Technology |          Jack | 2026-03-10 |    70 |        1 |   Shipped
       16 |        Cyber Security |  Technology |         Frank | 2026-03-16 |    80 |        1 |   Pending
       13 |            C++ Basics | Programming |       Charlie | 2026-03-13 |    30 |        2 | Delivered
       11 |      Machine Learning |  Technology |         Alice | 2026-03-11 |    65 |        1 | Delivered
        1 |      Redis Essentials |    Database |         Alice | 2026-03-01 | 29.99 |        1 | Delivered
       19 |           Data Mining |    Database |           Ivy | 2026-03-19 |    60 |        1 | Delivered
        8 |       Cloud Computing |  Technology |         Henry | 2026-03-08 |    55 |        1 |   Pending
        2 |        Learning NoSQL |    Database |           Bob | 2026-03-02 |  35.5 |        2 |   Shipped
        4 |    Python Programming | Programming |         David | 2026-03-04 |    40 |        1 | Delivered
       18 | Networking Essentials |  Technology |         Henry | 2026-03-18 |    50 |        2 |   Shipped
       15 |       Web Development | Programming |          Emma | 2026-03-15 |    45 |        2 | Delivered
       22 |       Data Structures | Programming |           Bob | 2026-03-22 |    40 |        2 | Delivered
       20 |     Operating Systems |  Technology |          Jack | 2026-03-20 |    55 |        2 | Cancelled
        7 |             AI Basics |  Technology |         Grace | 2026-03-07 |    60 |        2 |   Shipped
        6 |    Data Science Intro |  Technology |         Frank | 2026-03-06 |    50 |        1 | Delivered
        9 |            SQL Mastery |    Database |           Ivy | 2026-03-09 |    38 |        2 | Delivered
       14 |      JavaScript Guide | Programming |         David | 2026-03-14 |    35 |        1 |   Shipped
       21 |            Algorithms | Programming |         Alice | 2026-03-21 |    45 |        1 | Delivered
       17 |     Blockchain Basics |  Technology |         Grace | 2026-03-17 |    90 |        1 | Delivered
       12 |         Deep Learning |  Technology |           Bob | 2026-03-12 |    75 |        1 |   Pending
        3 |        MongoDB Basics |    Database |       Charlie | 2026-03-03 | 27.75 |        1 |   Pending

(22 rows)
cqlsh:bookstore>
```

Query 2 – Find specific order

```
cqlsh:bookstore> SELECT * FROM orders WHERE order_id = 5;

 order_id | book_title        | category    | customer_name | order_date | price | quantity | status
----------+-------------------+-------------+---------------+------------+-------+----------+-----------
        5 | Java Fundamentals | Programming |          Emma | 2026-03-05 |    45 |        3 | Cancelled

(1 rows)
```

Query 3 – Filter by customer

```
cqlsh:bookstore> SELECT * FROM orders WHERE customer_name = 'Alice' ALLOW FILTERING;

 order_id | book_title       | category    | customer_name | order_date | price | quantity | status
----------+------------------+-------------+---------------+------------+-------+----------+-----------
       11 | Machine Learning |  Technology |         Alice | 2026-03-11 |    65 |        1 | Delivered
        1 | Redis Essentials |    Database |         Alice | 2026-03-01 | 29.99 |        1 | Delivered
       21 |       Algorithms | Programming |         Alice | 2026-03-21 |    45 |        1 | Delivered

(3 rows)
```

Query 4 – Filter by category

```
cqlsh:bookstore> SELECT * FROM orders WHERE category = 'Programming' ALLOW FILTERING;

 order_id | book_title          | category    | customer_name | order_date | price | quantity | status
----------+---------------------+-------------+---------------+------------+-------+----------+-----------
        5 |  Java Fundamentals  | Programming |          Emma | 2026-03-05 |    45 |        3 | Cancelled
       13 |          C++ Basics | Programming |       Charlie | 2026-03-13 |    30 |        2 | Delivered
        4 | Python Programming  | Programming |         David | 2026-03-04 |    40 |        1 | Delivered
       15 |     Web Development | Programming |          Emma | 2026-03-15 |    45 |        2 | Delivered
       22 |     Data Structures | Programming |           Bob | 2026-03-22 |    40 |        2 | Delivered
       14 |    JavaScript Guide | Programming |         David | 2026-03-14 |    35 |        1 |   Shipped
       21 |          Algorithms | Programming |         Alice | 2026-03-21 |    45 |        1 | Delivered

(7 rows)
```

Query 5 – Filter by status

```
cqlsh:bookstore> SELECT * FROM orders WHERE status = 'Delivered' ALLOW FILTERING;

 order_id | book_title          | category    | customer_name | order_date | price | quantity | status
----------+---------------------+-------------+---------------+------------+-------+----------+-----------
       13 |          C++ Basics | Programming |       Charlie | 2026-03-13 |    30 |        2 | Delivered
       11 |    Machine Learning |  Technology |         Alice | 2026-03-11 |    65 |        1 | Delivered
        1 |    Redis Essentials |    Database |         Alice | 2026-03-01 | 29.99 |        1 | Delivered
       19 |         Data Mining |    Database |           Ivy | 2026-03-19 |    60 |        1 | Delivered
        4 | Python Programming  | Programming |         David | 2026-03-04 |    40 |        1 | Delivered
       15 |     Web Development | Programming |          Emma | 2026-03-15 |    45 |        2 | Delivered
       22 |     Data Structures | Programming |           Bob | 2026-03-22 |    40 |        2 | Delivered
        6 | Data Science Intro  |  Technology |         Frank | 2026-03-06 |    50 |        1 | Delivered
        9 |         SQL Mastery |    Database |           Ivy | 2026-03-09 |    38 |        2 | Delivered
       21 |          Algorithms | Programming |         Alice | 2026-03-21 |    45 |        1 | Delivered
       17 |  Blockchain Basics  |  Technology |         Grace | 2026-03-17 |    90 |        1 | Delivered

(11 rows)
```

Query 6 – Orders with price > 50

```
cqlsh:bookstore> SELECT * FROM orders WHERE price > 50 ALLOW FILTERING;

 order_id | book_title          | category    | customer_name | order_date | price | quantity | status
----------+---------------------+-------------+---------------+------------+-------+----------+-----------
       10 | Big Data Analytics  | Technology  |          Jack | 2026-03-10 |    70 |        1 |   Shipped
       16 |      Cyber Security | Technology  |         Frank | 2026-03-16 |    80 |        1 |   Pending
       11 |    Machine Learning | Technology  |         Alice | 2026-03-11 |    65 |        1 | Delivered
       19 |         Data Mining |   Database  |           Ivy | 2026-03-19 |    60 |        1 | Delivered
        8 |     Cloud Computing | Technology  |         Henry | 2026-03-08 |    55 |        1 |   Pending
       20 |  Operating Systems  | Technology  |          Jack | 2026-03-20 |    55 |        2 | Cancelled
        7 |           AI Basics | Technology  |         Grace | 2026-03-07 |    60 |        2 |   Shipped
       17 |  Blockchain Basics  | Technology  |         Grace | 2026-03-17 |    90 |        1 | Delivered
       12 |       Deep Learning | Technology  |           Bob | 2026-03-12 |    75 |        1 |   Pending

(9 rows)
```

Query 7 – Count total orders

```
cqlsh:bookstore> SELECT COUNT(*) FROM orders;

 count
-------
    22

(1 rows)
```

Query 8 – Update order status

```
cqlsh:bookstore> UPDATE orders SET status = 'Delivered' WHERE order_id = 3;
cqlsh:bookstore>
```

Query 9 – Delete an order

```
cqlsh:bookstore> DELETE FROM orders WHERE order_id = 20;
cqlsh:bookstore>
```

Query 10 – Orders by date

```
cqlsh:bookstore> SELECT * FROM orders WHERE order_date = '2026-03-10' ALLOW FILTERING;

 order_id | book_title         | category   | customer_name | order_date | price | quantity | status
----------+--------------------+------------+---------------+------------+-------+----------+---------
       10 | Big Data Analytics | Technology |          Jack | 2026-03-10 |    70 |        1 | Shipped

(1 rows)
```

Explanation

A Cassandra database was set up to handle bookstore orders. First, a keyspace called “bookstore” was created. Then, a column-family (table) named “orders” was added, which includes 8 attributes such as order details, customer information, pricing, and status.

The dataset has 30 records, meeting the coursework requirement. Queries were crafted to show data retrieval, filtering, updating, and deletion. Cassandra’s distributed structure allows for efficient management of large-scale structured data. The use of ALLOW FILTERING shows flexibility in querying, though it can affect performance with large datasets.

Design Rationale

- Primary key: order_id ensures uniqueness
- Flat table design → optimized for fast reads
- No joins → Cassandra best practice
- Data duplication avoided

Benefits

- High scalability
- Fast writes
- Distributed system
- Fault tolerance

Drawbacks

- Limited complex queries
- No joins
- ALLOW FILTERING can be inefficient

Assignment 3 – JSON Document & Schema

Purpose and Document Design

The goal of this assignment is to create a JSON document that represents an online bookstore system. This document models important parts like the store, customers, and orders in a clear, structured format.

The JSON document follows a nested design. The main object includes the store, customers, and orders. The store object holds general information. The customers section is an array that contains multiple customer records. The orders array has order details and a nested books array for purchased items.

Various data types are used, such as strings, numbers, booleans, arrays, and objects. This design shows how JSON can effectively represent complex, real-world data in a flexible way.

Schema Design and Validation

A JSON schema was created to define and validate the structure of the document. The schema specifies required fields, data types, and constraints for each part of the JSON.

The root object contains three required sections: store, customers, and orders. Each section has specific properties with defined data types to ensure consistency. The schema also validates nested objects and arrays.

Validation was done using an online tool like JSONLint. The results confirmed that the JSON document is valid and follows the defined schema.

This validation ensures data accuracy, consistency, and reliability.

Discussion

The implementation shows how JSON can be used to represent complex data through nested data structures. This means that related data, such as books in orders, can be grouped together, making it more readable

Benefits

- Flexible and easy to alter
- Supports complex nested data
- Schema validation for better data quality

Drawback

- Complex due to nested data
- No relationships, as in relational database

Overall

The JSON document and schema successfully demonstrate structured data design, validation, and flexibility, meeting the requirements of the assignment.

Assignment 4 – MongoDB Implementation

Case Study: Online Bookstore System

This assignment demonstrates the use of MongoDB to store and manage bookstore data using JSON-like documents.

Start MongoDB

Open terminal:

```
[nosql@nosql ~]$ mongo
MongoDB shell version v4.4.16
connecting to: mongodb://127.0.0.1:27017/?compressors=disabled&gssapiServiceNam
e=mongodb
Implicit session: session { "id" : UUID("00fb14f5-7cce-4ee4-92bd-18646fbecc5a")
 }
MongoDB server version: 4.4.16
Welcome to the MongoDB shell.
For interactive help, type "help".
For more comprehensive documentation, see
        https://docs.mongodb.com/
Questions? Try the MongoDB Developer Community Forums
        https://community.mongodb.com
---
The server generated these startup warnings when booting:
        2026-01-29T09:50:01.706+00:00: Access control is not enabled for the da
tabase. Read and write access to data and configuration is unrestricted
        2026-01-29T09:50:01.706+00:00: /sys/kernel/mm/transparent_hugepage/enab
led is 'always'. We suggest setting it to 'never'
        2026-01-29T09:50:01.706+00:00: /sys/kernel/mm/transparent_hugepage/defr
```

Create Database

```
> use bookstore
switched to db bookstore
>
```

Create Collection & Insert Data

```
> db.orders.insertMany([
... {order_id:1, customer:"Alice", book:"Redis Essentials", category:"Database", price:29.99, quantity:1, status:"Delivered"},
... {order_id:2, customer:"Bob", book:"Learning NoSQL", category:"Database", price:35.50, quantity:2, status:"Shipped"},
... {order_id:3, customer:"Charlie", book:"MongoDB Basics", category:"Database", price:27.75, quantity:1, status:"Pending"},
... {order_id:4, customer:"David", book:"Python Programming", category:"Programming", price:40.00, quantity:1, status:"Delivered"},
... {order_id:5, customer:"Emma", book:"Java Fundamentals", category:"Programming", price:45.00, quantity:3, status:"Cancelled"},
...
... {order_id:6, customer:"Frank", book:"Data Science Intro", category:"Technology", price:50.00, quantity:1, status:"Delivered"},
... {order_id:7, customer:"Grace", book:"AI Basics", category:"Technology", price:60.00, quantity:2, status:"Shipped"},
... {order_id:8, customer:"Henry", book:"Cloud Computing", category:"Technology", price:55.00, quantity:1, status:"Pending"},
... {order_id:9, customer:"Ivy", book:"SQL Mastery", category:"Database", price:38.00, quantity:2, status:"Delivered"},
... {order_id:10, customer:"Jack", book:"Big Data Analytics", category:"Technology", price:70.00, quantity:1, status:"Shipped"},
...
... {order_id:11, customer:"Alice", book:"Machine Learning", category:"Technology", price:65.00, quantity:1, status:"Delivered"},
... {order_id:12, customer:"Bob", book:"Deep Learning", category:"Technology", price:75.00, quantity:1, status:"Pending"},
... {order_id:13, customer:"Charlie", book:"C++ Basics", category:"Programming", price:30.00, quantity:2, status:"Delivered"},
... {order_id:14, customer:"David", book:"JavaScript Guide", category:"Programming", price:35.00, quantity:1, status:"Shipped"},
... {order_id:15, customer:"Emma", book:"Web Development", category:"Programming", price:45.00, quantity:2, status:"Delivered"},
...
... {order_id:16, customer:"Frank", book:"Cyber Security", category:"Technology", price:80.00, quantity:1, status:"Pending"},
... {order_id:17, customer:"Grace", book:"Blockchain Basics", category:"Technology", price:90.00, quantity:1, status:"Delivered"},
... {order_id:18, customer:"Henry", book:"Networking Essentials", category:"Technology", price:50.00, quantity:2, status:"Shipped"},
... {order_id:19, customer:"Ivy", book:"Data Mining", category:"Database", price:60.00, quantity:1, status:"Delivered"},
... {order_id:20, customer:"Jack", book:"Operating Systems", category:"Technology", price:55.00, quantity:2, status:"Cancelled"},
...
... {order_id:21, customer:"Alice", book:"Algorithms", category:"Programming", price:45.00, quantity:1, status:"Delivered"},
... {order_id:22, customer:"Bob", book:"Data Structures", category:"Programming", price:40.00, quantity:2, status:"Delivered"},
... {order_id:23, customer:"Charlie", book:"Linux Basics", category:"Technology", price:35.00, quantity:1, status:"Pending"},
... {order_id:24, customer:"David", book:"DevOps Guide", category:"Technology", price:60.00, quantity:1, status:"Delivered"},
... {order_id:25, customer:"Emma", book:"Software Engineering", category:"Programming", price:50.00, quantity:2, status:"Shipped"},
...
... {order_id:26, customer:"Frank", book:"Kubernetes", category:"Technology", price:70.00, quantity:1, status:"Delivered"},
... {order_id:27, customer:"Grace", book:"Docker Essentials", category:"Technology", price:65.00, quantity:2, status:"Delivered"},
... {order_id:28, customer:"Henry", book:"React Guide", category:"Programming", price:55.00, quantity:1, status:"Pending"},
... {order_id:29, customer:"Ivy", book:"Angular Basics", category:"Programming", price:50.00, quantity:1, status:"Delivered"},
... {order_id:30, customer:"Jack", book:"Vue.js Intro", category:"Programming", price:45.00, quantity:2, status:"Delivered"}
... ])
{
```

```
        "acknowledged" : true,
        "insertedIds" : [
                ObjectId("69bba86b99a7e85383b32ccf"),
                ObjectId("69bba86b99a7e85383b32cd0"),
                ObjectId("69bba86b99a7e85383b32cd1"),
                ObjectId("69bba86b99a7e85383b32cd2"),
                ObjectId("69bba86b99a7e85383b32cd3"),
                ObjectId("69bba86b99a7e85383b32cd4"),
                ObjectId("69bba86b99a7e85383b32cd5"),
                ObjectId("69bba86b99a7e85383b32cd6"),
                ObjectId("69bba86b99a7e85383b32cd7"),
                ObjectId("69bba86b99a7e85383b32cd8"),
                ObjectId("69bba86b99a7e85383b32cd9"),
                ObjectId("69bba86b99a7e85383b32cda"),
                ObjectId("69bba86b99a7e85383b32cdb"),
                ObjectId("69bba86b99a7e85383b32cdc"),
                ObjectId("69bba86b99a7e85383b32cdd"),
                ObjectId("69bba86b99a7e85383b32cde"),
                ObjectId("69bba86b99a7e85383b32cdf"),
                ObjectId("69bba86b99a7e85383b32ce0"),
                ObjectId("69bba86b99a7e85383b32ce1"),
                ObjectId("69bba86b99a7e85383b32ce2"),
                ObjectId("69bba86b99a7e85383b32ce3"),
                ObjectId("69bba86b99a7e85383b32ce4"),
                ObjectId("69bba86b99a7e85383b32ce5"),
                ObjectId("69bba86b99a7e85383b32ce6"),
                ObjectId("69bba86b99a7e85383b32ce7"),
                ObjectId("69bba86b99a7e85383b32ce8"),
                ObjectId("69bba86b99a7e85383b32ce9"),
                ObjectId("69bba86b99a7e85383b32cea"),
                ObjectId("69bba86b99a7e85383b32ceb"),
                ObjectId("69bba86b99a7e85383b32cec")
        ]
}
>
```

Verify Data

db.orders.countDocuments()

```
> db.orders.countDocuments()
uncaught exception: Error: command failed: {
        "ok" : 0,
        "errmsg" : "the match filter must be an expression in an object",
        "code" : 15959,
        "codeName" : "Location15959"
} : aggregate failed :
_getErrorWithCode@src/mongo/shell/utils.js:25:13
doassert@src/mongo/shell/assert.js:18:14
_assertCommandWorked@src/mongo/shell/assert.js:705:17
assert.commandWorked@src/mongo/shell/assert.js:795:16
DB.prototype._runAggregate@src/mongo/shell/db.js:266:5
DBCollection.prototype.aggregate@src/mongo/shell/collection.js:1058:12
DBCollection.prototype.countDocuments@src/mongo/shell/collection.js:1450:17
@(shell):1:1
>
```

Queries

Query 1 – View all

db.orders.find ()

```
> db.orders.insertMany([
... {order_id:1, customer:"Alice", book:"Redis Essentials", category:"Database", price:29.99, quantity:1, status:"Delivered"},
... {order_id:2, customer:"Bob", book:"Learning NoSQL", category:"Database", price:35.50, quantity:2, status:"Shipped"},
... {order_id:3, customer:"Charlie", book:"MongoDB Basics", category:"Database", price:27.75, quantity:1, status:"Pending"},
... {order_id:4, customer:"David", book:"Python Programming", category:"Programming", price:40.00, quantity:1, status:"Delivered"},
... {order_id:5, customer:"Emma", book:"Java Fundamentals", category:"Programming", price:45.00, quantity:3, status:"Cancelled"},
...
... {order_id:6, customer:"Frank", book:"Data Science Intro", category:"Technology", price:50.00, quantity:1, status:"Delivered"},
... {order_id:7, customer:"Grace", book:"AI Basics", category:"Technology", price:60.00, quantity:2, status:"Shipped"},
... {order_id:8, customer:"Henry", book:"Cloud Computing", category:"Technology", price:55.00, quantity:1, status:"Pending"},
... {order_id:9, customer:"Ivy", book:"SQL Mastery", category:"Database", price:38.00, quantity:2, status:"Delivered"},
... {order_id:10, customer:"Jack", book:"Big Data Analytics", category:"Technology", price:70.00, quantity:1, status:"Shipped"},
...
... {order_id:11, customer:"Alice", book:"Machine Learning", category:"Technology", price:65.00, quantity:1, status:"Delivered"},
... {order_id:12, customer:"Bob", book:"Deep Learning", category:"Technology", price:75.00, quantity:1, status:"Pending"},
... {order_id:13, customer:"Charlie", book:"C++ Basics", category:"Programming", price:30.00, quantity:2, status:"Delivered"},
... {order_id:14, customer:"David", book:"JavaScript Guide", category:"Programming", price:35.00, quantity:1, status:"Shipped"},
... {order_id:15, customer:"Emma", book:"Web Development", category:"Programming", price:45.00, quantity:2, status:"Delivered"},
...
... {order_id:16, customer:"Frank", book:"Cyber Security", category:"Technology", price:80.00, quantity:1, status:"Pending"},
... {order_id:17, customer:"Grace", book:"Blockchain Basics", category:"Technology", price:90.00, quantity:1, status:"Delivered"},
... {order_id:18, customer:"Henry", book:"Networking Essentials", category:"Technology", price:50.00, quantity:2, status:"Shipped"},
... {order_id:19, customer:"Ivy", book:"Data Mining", category:"Database", price:60.00, quantity:1, status:"Delivered"},
... {order_id:20, customer:"Jack", book:"Operating Systems", category:"Technology", price:55.00, quantity:2, status:"Cancelled"},
...
... {order_id:21, customer:"Alice", book:"Algorithms", category:"Programming", price:45.00, quantity:1, status:"Delivered"},
... {order_id:22, customer:"Bob", book:"Data Structures", category:"Programming", price:40.00, quantity:2, status:"Delivered"},
... {order_id:23, customer:"Charlie", book:"Linux Basics", category:"Technology", price:35.00, quantity:1, status:"Pending"},
... {order_id:24, customer:"David", book:"DevOps Guide", category:"Technology", price:60.00, quantity:1, status:"Delivered"},
... {order_id:25, customer:"Emma", book:"Software Engineering", category:"Programming", price:50.00, quantity:2, status:"Shipped"},
...
... {order_id:26, customer:"Frank", book:"Kubernetes", category:"Technology", price:70.00, quantity:1, status:"Delivered"},
... {order_id:27, customer:"Grace", book:"Docker Essentials", category:"Technology", price:65.00, quantity:2, status:"Delivered"},
... {order_id:28, customer:"Henry", book:"React Guide", category:"Programming", price:55.00, quantity:1, status:"Pending"},
... {order_id:29, customer:"Ivy", book:"Angular Basics", category:"Programming", price:50.00, quantity:1, status:"Delivered"},
... {order_id:30, customer:"Jack", book:"Vue.js Intro", category:"Programming", price:45.00, quantity:2, status:"Delivered"}
... ])
{
```

Query 2 – Specific order

db.orders.find ({order_id:5})

```
        "acknowledged" : true,
        "insertedIds" : [
                ObjectId("69bba86b99a7e85383b32ccf"),
                ObjectId("69bba86b99a7e85383b32cd0"),
                ObjectId("69bba86b99a7e85383b32cd1"),
                ObjectId("69bba86b99a7e85383b32cd2"),
                ObjectId("69bba86b99a7e85383b32cd3"),
                ObjectId("69bba86b99a7e85383b32cd4"),
                ObjectId("69bba86b99a7e85383b32cd5"),
                ObjectId("69bba86b99a7e85383b32cd6"),
                ObjectId("69bba86b99a7e85383b32cd7"),
                ObjectId("69bba86b99a7e85383b32cd8"),
                ObjectId("69bba86b99a7e85383b32cd9"),
                ObjectId("69bba86b99a7e85383b32cda"),
                ObjectId("69bba86b99a7e85383b32cdb"),
                ObjectId("69bba86b99a7e85383b32cdc"),
                ObjectId("69bba86b99a7e85383b32cdd"),
                ObjectId("69bba86b99a7e85383b32cde"),
                ObjectId("69bba86b99a7e85383b32cdf"),
                ObjectId("69bba86b99a7e85383b32ce0"),
                ObjectId("69bba86b99a7e85383b32ce1"),
                ObjectId("69bba86b99a7e85383b32ce2"),
                ObjectId("69bba86b99a7e85383b32ce3"),
                ObjectId("69bba86b99a7e85383b32ce4"),
                ObjectId("69bba86b99a7e85383b32ce5"),
                ObjectId("69bba86b99a7e85383b32ce6"),
                ObjectId("69bba86b99a7e85383b32ce7"),
                ObjectId("69bba86b99a7e85383b32ce8"),
                ObjectId("69bba86b99a7e85383b32ce9"),
                ObjectId("69bba86b99a7e85383b32cea"),
                ObjectId("69bba86b99a7e85383b32ceb"),
                ObjectId("69bba86b99a7e85383b32cec")
        ]
}
>
```

Query 3 – Filter by customer

db.orders.find ({customer:"Alice"})

```
> db.orders.find({customer:"Alice"})
{ "_id" : ObjectId("69bba86b99a7e85383b32ccf"), "order_id" : 1, "customer" : "A
lice", "book" : "Redis Essentials", "category" : "Database", "price" : 29.99, "
quantity" : 1, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cd9"), "order_id" : 11, "customer" : "
Alice", "book" : "Machine Learning", "category" : "Technology", "price" : 65, "
quantity" : 1, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32ce3"), "order_id" : 21, "customer" : "
Alice", "book" : "Algorithms", "category" : "Programming", "price" : 45, "quant
ity" : 1, "status" : "Delivered" }
>
```

Query 4 – Price greater than 50

db.orders.find ({price:{$gt:50}})

```
> db.orders.find({price:{$gt:50}})
{ "_id" : ObjectId("69bba86b99a7e85383b32cd5"), "order_id" : 7, "customer" : "Grace", "book" : "AI Basics", "category" : "Technology", "price" : 60, "quantity" :
 2, "status" : "Shipped" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cd6"), "order_id" : 8, "customer" : "Henry", "book" : "Cloud Computing", "category" : "Technology", "price" : 55, "quant
ity" : 1, "status" : "Pending" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cd8"), "order_id" : 10, "customer" : "Jack", "book" : "Big Data Analytics", "category" : "Technology", "price" : 70, "qu
antity" : 1, "status" : "Shipped" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cd9"), "order_id" : 11, "customer" : "Alice", "book" : "Machine Learning", "category" : "Technology", "price" : 65, "qua
ntity" : 1, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cda"), "order_id" : 12, "customer" : "Bob", "book" : "Deep Learning", "category" : "Technology", "price" : 75, "quantity
" : 1, "status" : "Pending" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cde"), "order_id" : 16, "customer" : "Frank", "book" : "Cyber Security", "category" : "Technology", "price" : 80, "quant
ity" : 1, "status" : "Pending" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cdf"), "order_id" : 17, "customer" : "Grace", "book" : "Blockchain Basics", "category" : "Technology", "price" : 90, "qu
antity" : 1, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32ce1"), "order_id" : 19, "customer" : "Ivy", "book" : "Data Mining", "category" : "Database", "price" : 60, "quantity" :
1, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32ce2"), "order_id" : 20, "customer" : "Jack", "book" : "Operating Systems", "category" : "Technology", "price" : 55, "qua
ntity" : 2, "status" : "Cancelled" }
{ "_id" : ObjectId("69bba86b99a7e85383b32ce6"), "order_id" : 24, "customer" : "David", "book" : "DevOps Guide", "category" : "Technology", "price" : 60, "quantit
y" : 1, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32ce8"), "order_id" : 26, "customer" : "Frank", "book" : "Kubernetes", "category" : "Technology", "price" : 70, "quantity"
 : 1, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32ce9"), "order_id" : 27, "customer" : "Grace", "book" : "Docker Essentials", "category" : "Technology", "price" : 65, "qu
antity" : 2, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cea"), "order_id" : 28, "customer" : "Henry", "book" : "React Guide", "category" : "Programming", "price" : 55, "quantit
y" : 1, "status" : "Pending" }
>
```

Query 5 – AND condition

db.orders.find({category:"Programming", status:"Delivered"})

```
> db.orders.find({category:"Programming", status:"Delivered"})
{ "_id" : ObjectId("69bba86b99a7e85383b32cd2"), "order_id" : 4, "customer" : "David", "book" : "Python Programming", "category" : "Programming", "price" : 40, "q
uantity" : 1, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cdb"), "order_id" : 13, "customer" : "Charlie", "book" : "C++ Basics", "category" : "Programming", "price" : 30, "quanti
ty" : 2, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cdd"), "order_id" : 15, "customer" : "Emma", "book" : "Web Development", "category" : "Programming", "price" : 45, "quan
tity" : 2, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32ce3"), "order_id" : 21, "customer" : "Alice", "book" : "Algorithms", "category" : "Programming", "price" : 45, "quantity
" : 1, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32ce4"), "order_id" : 22, "customer" : "Bob", "book" : "Data Structures", "category" : "Programming", "price" : 40, "quant
ity" : 2, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32ceb"), "order_id" : 29, "customer" : "Ivy", "book" : "Angular Basics", "category" : "Programming", "price" : 50, "quanti
ty" : 1, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cec"), "order_id" : 30, "customer" : "Jack", "book" : "Vue.js Intro", "category" : "Programming", "price" : 45, "quantit
y" : 2, "status" : "Delivered" }
>
```

Query 6 – OR condition

db.orders.find({$or:[{status:"Pending"},{status:"Cancelled"}]})

```
> db.orders.find({$or:[{status:"Pending"},{status:"Cancelled"}]})
{ "_id" : ObjectId("69bba86b99a7e85383b32cd1"), "order_id" : 3, "customer" : "Charlie", "book" : "MongoDB Basics", "category" : "Database", "price" : 27.75, "qua
ntity" : 1, "status" : "Pending" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cd3"), "order_id" : 5, "customer" : "Emma", "book" : "Java Fundamentals", "category" : "Programming", "price" : 45, "qua
ntity" : 3, "status" : "Cancelled" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cd6"), "order_id" : 8, "customer" : "Henry", "book" : "Cloud Computing", "category" : "Technology", "price" : 55, "quant
ity" : 1, "status" : "Pending" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cda"), "order_id" : 12, "customer" : "Bob", "book" : "Deep Learning", "category" : "Technology", "price" : 75, "quantity
" : 1, "status" : "Pending" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cde"), "order_id" : 16, "customer" : "Frank", "book" : "Cyber Security", "category" : "Technology", "price" : 80, "quant
ity" : 1, "status" : "Pending" }
{ "_id" : ObjectId("69bba86b99a7e85383b32ce2"), "order_id" : 20, "customer" : "Jack", "book" : "Operating Systems", "category" : "Technology", "price" : 55, "qua
ntity" : 2, "status" : "Cancelled" }
{ "_id" : ObjectId("69bba86b99a7e85383b32ce5"), "order_id" : 23, "customer" : "Charlie", "book" : "Linux Basics", "category" : "Technology", "price" : 35, "quant
ity" : 1, "status" : "Pending" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cea"), "order_id" : 28, "customer" : "Henry", "book" : "React Guide", "category" : "Programming", "price" : 55, "quantit
y" : 1, "status" : "Pending" }
>
```

Query 7 – Update

db.orders.updateOne({order_id:3},{$set:{status:"Delivered"}})

```
> db.orders.updateOne({order_id:3},{$set:{status:"Delivered"}})
{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
>
```

Query 8 – Delete

db.orders.deleteOne({order_id:20})

```
> db.orders.updateOne({order_id:3},{$set:{status:"Delivered"}})
{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
> db.orders.updateOne({order_id:3},{$set:{status:"Delivered"}})
{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 0 }
>
```

Query 9 – Sort

db.orders.find().sort({price:-1})

```
> db.orders.find().sort({price:-1})
{ "_id" : ObjectId("69bba86b99a7e85383b32cdf"), "order_id" : 17, "customer" : "Grace", "book" : "Blockchain Basics", "category" : "Technology", "price" : 90, "qu
antity" : 1, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cde"), "order_id" : 16, "customer" : "Frank", "book" : "Cyber Security", "category" : "Technology", "price" : 80, "quant
ity" : 1, "status" : "Pending" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cda"), "order_id" : 12, "customer" : "Bob", "book" : "Deep Learning", "category" : "Technology", "price" : 75, "quantity
" : 1, "status" : "Pending" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cd8"), "order_id" : 10, "customer" : "Jack", "book" : "Big Data Analytics", "category" : "Technology", "price" : 70, "qu
antity" : 1, "status" : "Shipped" }
{ "_id" : ObjectId("69bba86b99a7e85383b32ce8"), "order_id" : 26, "customer" : "Frank", "book" : "Kubernetes", "category" : "Technology", "price" : 70, "quantity"
 : 1, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cd9"), "order_id" : 11, "customer" : "Alice", "book" : "Machine Learning", "category" : "Technology", "price" : 65, "qua
ntity" : 1, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32ce9"), "order_id" : 27, "customer" : "Grace", "book" : "Docker Essentials", "category" : "Technology", "price" : 65, "qu
antity" : 2, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cd5"), "order_id" : 7, "customer" : "Grace", "book" : "AI Basics", "category" : "Technology", "price" : 60, "quantity" :
 2, "status" : "Shipped" }
{ "_id" : ObjectId("69bba86b99a7e85383b32ce1"), "order_id" : 19, "customer" : "Ivy", "book" : "Data Mining", "category" : "Database", "price" : 60, "quantity" :
1, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32ce6"), "order_id" : 24, "customer" : "David", "book" : "DevOps Guide", "category" : "Technology", "price" : 60, "quantit
y" : 1, "status" : "Delivered" }
```

Query 10 – Limit

db.orders.find().limit(5)

```
> db.orders.find().limit(5)
{ "_id" : ObjectId("69bba86b99a7e85383b32ccf"), "order_id" : 1, "customer" : "A
lice", "book" : "Redis Essentials", "category" : "Database", "price" : 29.99, "
quantity" : 1, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cd0"), "order_id" : 2, "customer" : "B
ob", "book" : "Learning NoSQL", "category" : "Database", "price" : 35.5, "quant
ity" : 2, "status" : "Shipped" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cd1"), "order_id" : 3, "customer" : "C
harlie", "book" : "MongoDB Basics", "category" : "Database", "price" : 27.75, "
quantity" : 1, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cd2"), "order_id" : 4, "customer" : "D
avid", "book" : "Python Programming", "category" : "Programming", "price" : 40,
 "quantity" : 1, "status" : "Delivered" }
{ "_id" : ObjectId("69bba86b99a7e85383b32cd3"), "order_id" : 5, "customer" : "E
mma", "book" : "Java Fundamentals", "category" : "Programming", "price" : 45, "
quantity" : 3, "status" : "Cancelled" }
>
```

Purpose

The purpose of this assignment is to show how MongoDB can be used for storing and managing data in a NoSQL database system. Unlike relational database systems, MongoDB stores data in a flexible JSON-like format, referred to as documents, which makes it efficient for handling unstructured data.

For this assignment, an online bookstore system is created to mimic real-world data such as customers, books, and orders.

Implementation

A database called bookstore was created. A collection called orders was used to store the data. The data was stored in the collection using the insertMany() method in the form of documents. Each document represents a single order.

Each document consists of fields such as:

- order_id
- customer
- book
- category
- price
- quantity
- status

This shows that MongoDB is schema-less because documents can be stored without having to adhere to any structure.

Query Operations

Various queries were performed to demonstrate MongoDB functionality:

- **Data Retrieval:** Using find() to display all records
- **Filtering:** Queries based on customer name, category, and status
- **Conditional Queries:** Using operators like $gt (greater than) and $or
- **Update Operation:** Modifying existing data using $set
- **Delete Operation:** Removing records using deleteOne()
- **Sorting & Limiting:** Organising and restricting output using sort() and limit()

These operations show how MongoDB efficiently handles CRUD (Create, Read, Update, Delete) operations.

4. Advantages

- Flexible schema allows easy data modification
- High performance for large datasets
- Supports complex queries and indexing
- Scalable and suitable for distributed systems

5. Limitations

- No strong schema enforcement by default
- Data redundancy may occur
- Limited support for joins compared to relational databases

6. Conclusion

The assignment successfully demonstrates the implementation of a NoSQL database using MongoDB. The use of document-based storage, flexible schema, and powerful query operations highlights MongoDB's suitability for modern applications dealing with large and dynamic datasets.

Appendix

JSON DOCUMENT


```
{
 "store": {
  "name": "City Online Bookstore",
  "location": "London",
  "open": true,
  "rating": 4.7
 },
 "customers": [
  {
   "customer_id": 1,
   "name": "Alice",
   "email": "alice@example.com",
   "premium": true
  },
  {
   "customer_id": 2,
   "name": "Bob",
   "email": "bob@example.com",
   "premium": false
  }
 ],
 "orders": [
  {
   "order_id": 101,
   "customer_id": 1,
   "order_date": "2026-03-01",
   "total_price": 59.98,
   "books": [
    {
     "book_id": 1,
     "title": "Redis Essentials",
     "price": 29.99,
     "quantity": 2
    }
   ]
```

```
    },
    {
      "order_id": 102,
      "customer_id": 2,
      "order_date": "2026-03-02",
      "total_price": 35.50,
      "books": [
        {
          "book_id": 2,
          "title": "Learning NoSQL",
          "price": 35.50,
          "quantity": 1
        }
      ]
    }
  ]
}
```


JSON Schema


```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "required": ["store", "customers", "orders"],
  "properties": {
    "store": {
      "type": "object",
      "required": ["name", "location", "open", "rating"],
      "properties": {
        "name": { "type": "string" },
        "location": { "type": "string" },
        "open": { "type": "boolean" },
        "rating": { "type": "number" }
      }
    },
    "customers": {
      "type": "array",
```

SCREENSHOT

Fig: JSON_Schema_Validation_Success.